

Integration testing

COMS 4156

What is Integration testing?

- Integration testing is a phase of software testing in which individual units or components are combined and tested as a group. The goal is to identify issues that occur when units interact with each other.

Why is integration testing important?

- **Identify Issues Early:** Integration testing helps catch defects related to component interactions early in the development cycle.
- **Improves Reliability:** It ensures that data flows between components as expected, improving the overall reliability of the application.
- **Reduces Risk:** By verifying interactions between components, integration testing helps reduce the risk of major issues in production.

Differences between unit testing and integration testing

- **Focus on Component Interactions:** Unlike unit testing, which tests individual methods or classes in isolation, integration testing checks how multiple components interact.
- **Scope:** Integration tests are usually broader in scope than unit tests and might cover multiple classes, modules, or even subsystems.
- **Dependencies:** Integration tests often involve dependencies on external systems, databases, or APIs, whereas unit tests try to isolate the code as much as possible.

Approaches to Integration Testing

- **Top-Down Testing:** Start testing from the highest-level modules and progressively integrate lower-level modules.
- **Bottom-Up Testing:** Begin with lower-level modules and integrate upwards, testing as you go.
- **Big Bang Testing:** Integrate all components at once and test the entire system.
Mention that this can be risky and harder to debug.
- **Incremental Testing:** Gradually add and test modules in small increments, which can make isolating issues easier.

Best practices for integration testing in java

- **Use Dependency Injection:** Helps create testable code by allowing you to inject mock dependencies instead of relying on actual implementations.
- **Mock External Dependencies:** Use tools like Mockito to mock dependencies such as databases or external APIs.
- **Data Setup and Cleanup:** Properly set up test data before the tests run and clean up afterward to avoid side effects in subsequent tests.
- **Keep Tests Focused:** Aim for tests that focus on specific integrations rather than trying to cover everything in one go.
- **Automate Your Tests:** Use CI/CD tools to automate running integration tests as part of the build process.

Demo of integration test

- Basic integration test (walk through local code)
- [Integration tests with gpus](#) (python)

Resources

- [Tutorialspoint - Integration testing](#)
- [Lamba test - integration testing](#)